

Domanda 1

Dato il seguente codice:

```
C/C++
class Base {
protected:
    int x = 1;
public:
    void printX() { std::cout << x; }
};

class Derivata : private Base {
public:
    void accessX() {
        // Punto A
        std::cout << x;
    }
};
```

Qual è la **visibilità** del membro `x` (definito come `protected` in `Base`) all'interno del metodo `accessX()` della classe `Derivata`?

- A. `public`. È accessibile in modo normale.
- B. `protected`. È accessibile all'interno di `Derivata`, ma non da oggetti esterni a `Derivata`.
- C. `private`. La clausola `private` nell'ereditarietà rende tutti i membri ereditati `private` in `Derivata`.
- D. Non è accessibile, causerebbe un errore di compilazione.

Risposta Corretta: C

Nell'ereditarietà privata (`class Derivata : private Base`), tutti i membri di `Base` (pubblici e protetti) diventano `private` all'interno di `Derivata`. Anche se `x` è `protected` in `Base`, in `Derivata` diventa `private`, ma rimane comunque **accessibile** (un membro `private` è accessibile all'interno della sua stessa classe) come mostrato dal codice al Punto A.

Domanda 2

Data la seguente gerarchia:

```

C/C++
class Base {
protected:
    void method1() {}
};

class Derivata : protected Base {
public:
    void method2() { method1(); }
};

int main() {
    Derivata d;
    // Punto B
    // d.method1();
}

```

Cosa succede quando si tenta di chiamare `d.method1()` (Punto B) dall'esterno della classe `Derivata`?

- A. La chiamata è valida, poiché `method1` era `protected` in `Base`.
- B. La chiamata è valida, poiché `method2` è `public` e può accedervi.
- C. Errore di compilazione. `method1` diventa `protected` in `Derivata` e non è accessibile al di fuori della classe.
- D. Errore di compilazione, ma solo se `method1` fosse stato `private` in `Base`.

Risposta Corretta: C

Nell'ereditarietà protetta (`class Derivata : protected Base`), i membri `protected` e `public` di `Base` diventano `protected` in `Derivata`. Un membro `protected` non è accessibile da codice esterno (come in `main`), causando un errore di compilazione al Punto B.

Domanda 3:

```

C/C++
class A {
private:
    int value = 5;
};

```

```
class B : public A {
public:
    void printValue() {
        // Punto C
        // std::cout << value;
    }
};
```

Cosa si verifica quando la classe **B** tenta di accedere al membro **value** della classe **A** (Punto C)?

- A. Nessun problema, l'ereditarietà **public** consente l'accesso.
- B. L'accesso è consentito perché **value** è un membro della classe base.
- C. Errore di compilazione. I membri **private** di una classe base non sono mai accessibili direttamente nelle classi derivate.
- D. È necessario utilizzare la risoluzione dell'ambito (**A::value**) per accedervi.

Risposta Corretta: C

I membri dichiarati come **private** sono accessibili **solo all'interno della classe in cui sono definiti**. Le classi derivate (anche con ereditarietà **public**) non hanno accesso diretto ai membri **private** della loro classe base.

Domanda 4:

Date due classi **Base** e **Derivata** (**Derivata : public Base**), in quale ordine vengono eseguiti i loro costruttori e distruttori quando viene creato e poi distrutto un oggetto di tipo **Derivata**?

- A. Costruttore **Derivata**, Costruttore **Base**, Distruttore **Base**, Distruttore **Derivata**.
- B. Costruttore **Base**, Costruttore **Derivata**, Distruttore **Derivata**, Distruttore **Base**.
- C. Costruttore **Base**, Costruttore **Derivata**, Distruttore **Base**, Distruttore **Derivata**.
- D. Distruttore **Derivata**, Distruttore **Base**, Costruttore **Base**, Costruttore **Derivata**.

Risposta Corretta: B

I **costruttori** vengono chiamati dall'alto verso il basso (prima la Base, poi la Derivata). I **distruttori** vengono chiamati in ordine inverso (prima la Derivata, poi la Base).

Domanda 5:

```
C/C++
class A {
```

```
protected:
    int valA = 10;
};
class B : public A { /* ... */ };
class C : private B {
public:
    void printA() {
        // Punto E
        std::cout << valA;
    }
};
```

Il codice al Punto E è valido?

- A. No, l'ereditarietà tra C e B è private, quindi valA non è accessibile.
- B. No, valA è protected e non può mai essere stampato.
- C. Sì. In B, valA è protected. In C, a causa dell'ereditarietà private da B, valA diventa private in C, ma è comunque accessibile all'interno dei metodi di C.
- D. No, valA è definito troppo in alto nella gerarchia.

Risposta Corretta: C

Il percorso di ereditarietà è: valA (protected in A) -> valA (protected in B perché l'ereditarietà è public) -> valA (private in C perché l'ereditarietà è private). Poiché è private in C, è accessibile all'interno di printA().

Domanda 6:

In C++, un membro dichiarato **protected** è accessibile:

- A. Solo all'interno della classe in cui è definito.
- B. All'interno della classe, dalle classi derivate, e da codice esterno tramite puntatori/oggetti.
- C. All'interno della classe e dalle classi direttamente o indirettamente derivate.
- D. All'interno della classe, dalle classi derivate e da classi amiche.

Risposta Corretta: C

Un membro **protected** è accessibile **all'interno della sua classe e all'interno di qualsiasi classe che eredita** da essa (direttamente o indirettamente), indipendentemente dal tipo di ereditarietà (**public**, **protected**, **private**).

Domanda 7

```
C/C++
class A {
public:
    int x = 1;
};

class B : public A {
public:
    int x = 2;
    void print() {
        // Punto F
        std::cout << x << " " << A::x;
    }
};
```

Qual è l'output del metodo `print()` (Punto F) se chiamato su un oggetto `B`?

- A. 1 1
- B. 2 1
- C. 1 2
- D. 2 2

Risposta Corretta: B

Quando una classe derivata definisce un membro con lo stesso nome di un membro ereditato (`x` in questo caso), il membro ereditato viene **nascosto** (non *sovrascritto*). `x` fa riferimento al membro di `B` (2), mentre `A::x` forza l'accesso al membro della classe base (1).

Domanda 8

Dato il seguente codice:

```
C/C++
class Base {
public:
    void display() { std::cout << "Base"; }
};

class Derivata : public Base {
public:
```

```

    void display() { std::cout << "Derivata"; }
};

void check(Base& b) {
    b.display(); // Punto G
}

int main() {
    Derivata d;
    check(d);
}

```

Qual è l'output del codice al Punto G?

- A. **Derivata**
- B. **Base**
- C. Errore di compilazione
- D. Errore di runtime

Risposta Corretta: B

Senza la parola chiave **virtual** sulla funzione **display()** in **Base**, C++ utilizza il **binding statico**. La chiamata al metodo è determinata dal **tipo di riferimento** usato (**Base& b**), non dal tipo effettivo dell'oggetto (**Derivata**).

Domanda 9:

Riprendendo la domanda precedente, ma aggiungendo **virtual**:

```

C/C++
class Base {
public:
    virtual void display() { std::cout << "Base"; }
};
// ... Derivata come prima

void check(Base& b) {
    b.display(); // Punto H
}

```

```
// ... main come prima
```

Qual è l'output del codice al Punto H?

- A. **Base**
- B. **Derivata**
- C. Errore di compilazione
- D. Errore di runtime

Risposta Corretta: B

Con la parola chiave **virtual**, C++ utilizza il **binding dinamico** (polimorfismo). La chiamata al metodo è determinata dal **tipo effettivo dell'oggetto puntato** (o referenziato), che è **Derivata**, anche se il riferimento è di tipo **Base**.

Domanda 10

Perché è fondamentale che il distruttore della classe base sia dichiarato **virtual** quando si usa il polimorfismo (ovvero si cancella un oggetto derivato tramite un puntatore alla classe base)?

- A. Per garantire che venga chiamato solo il distruttore della classe base.
- B. Per impedire l'object slicing durante la distruzione.
- C. Per garantire che venga chiamato il distruttore della classe derivata e, successivamente, quello della classe base.
- D. I distruttori virtuali sono richiesti solo in presenza di funzioni virtuali pure.

Risposta Corretta: C

Se il distruttore della classe base non è **virtual**, l'istruzione **delete ptr_base;** (dove **ptr_base** punta a un oggetto derivato) causerà il **binding statico**, chiamando **solo** il distruttore di **Base**. Questo porta a una **perdita di risorse** (memory leak) perché il distruttore di **Derivata** non viene eseguito. Il distruttore **virtual** abilita il binding dinamico.

Domanda 11

Qual è lo scopo principale dello specificatore **override** (opzionale ma consigliato) in C++?

- A. Forzare la funzione a essere **virtual**.
- B. Dichiarare che la funzione può essere sovrascritta.
- C. Garantire al compilatore che il metodo sta effettivamente sovrascrivendo un metodo virtuale della classe base (e segnalare un errore in caso contrario).
- D. Rendere la funzione non sovrascrivibile in ulteriori classi derivate.

Risposta Corretta: C

override è un safety check. Aiuta a prevenire errori sottili (ad esempio, un errore di battitura nella firma della funzione) che farebbero sì che un programmatore pensi di star sovrascrivendo un metodo virtuale, quando in realtà sta solo **nascondendo** (hiding) un metodo della classe base.

Domanda 12

```
C/C++
class Figura {
public:
    virtual double area() = 0; // Punto I
};
```

Cosa implica l'uguaglianza `= 0` nella dichiarazione del metodo `area()` (Punto I)?

- A. La funzione restituisce zero.
- B. La funzione non può essere ridefinita.
- C. Rende `area()` una funzione virtuale pura e `Figura` una classe astratta, che non può essere istanziata.
- D. La funzione deve essere definita nel file sorgente (`.cpp`) e non nell'intestazione.

Risposta Corretta: C

Una funzione virtuale pura (`= 0`) rende la classe che la contiene una **classe astratta**. Una classe astratta serve solo come interfaccia e **non può essere istanziata** (cioè, non si possono creare oggetti di tipo `Figura`).

Domanda 13

Qual è la regola quando si chiama un metodo `virtual` all'interno del **costruttore** di una classe base?

- A. Viene eseguito il metodo della classe derivata se l'oggetto è in fase di costruzione.
- B. Causa un errore di compilazione.
- C. Viene sempre eseguito il metodo definito nella classe base stessa (binding statico).
- D. Viene eseguito il metodo della classe derivata se è dichiarato con **override**.

Risposta Corretta: C

Durante l'esecuzione del costruttore di una classe base, la parte derivata dell'oggetto non è ancora completamente costruita. Per evitare di operare su dati non inizializzati, il meccanismo di **polimorfismo è disabilitato**. Qualsiasi chiamata a un metodo virtuale da un costruttore o distruttore utilizza il **binding statico** (chiamando la versione del metodo definita nella classe attuale).

Domanda 14

Dato il seguente codice:

```
C/C++
class Base {
public:
    int b = 1;
};

class Derivata : public Base {
public:
    int d = 2;
};

int main() {
    Derivata d_obj;
    Base b_obj = d_obj; // Punto K: Assegnazione per valore

    // std::cout << b_obj.d; // Tentativo di accedere a 'd'
}
```

Cosa succede al Punto K (Assegnazione per valore) e quale porzione dell'oggetto `d_obj` viene copiata in `b_obj`?

- A. Viene copiata l'intera `d_obj` e `b_obj` contiene sia `b` che `d`.
- B. Si verifica l'Object Slicing: solo la porzione `Base` di `d_obj` (il membro `b`) viene copiata in `b_obj`, e il membro `d` viene "tagliato" (slicing).
- C. La classe `Base` deve avere un costruttore di copia per consentire l'assegnazione.
- D. Il codice al Punto K è valido, ma l'accesso a `b_obj.d` causerebbe un errore di runtime.

Risposta Corretta: B

Quando un oggetto di una classe derivata viene assegnato a un oggetto di una classe base per **valore**, viene copiata solo la **sottoparte Base** dell'oggetto. La parte Derivata viene scartata (o *affettata*, in inglese *sliced*). Questo è chiamato **Object Slicing**.

Domanda 15

Dato il seguente codice:

C/C++

```
class A { /* ... */ };  
class B : public A { /* ... */ };  
class C { /* ... */ };  
  
int main() {  
    B b_obj;  
    C c_obj;  
  
    A* ptr1 = &b_obj; // Linea 1  
    B* ptr2 = &b_obj; // Linea 2  
    A* ptr3 = &c_obj; // Linea 3  
}
```

Quale riga causerà un **errore di compilazione**?

- A. Linea 1
- B. Linea 2
- C. Linea 3
- D. Nessuna

Risposta Corretta: C

La **Linea 3** causerà un errore perché la classe **C non è correlata** ad **A** tramite ereditarietà. Non è possibile assegnare un puntatore a un oggetto di tipo **C** a un puntatore di tipo **A**. Le Linee 1 e 2 sono valide (la Linea 1 è un esempio di **Upcasting**).

Domanda 16

C/C++

```
class Base { /* ... */ };  
class Derivata : public Base {  
public:  
    void special() {}  
};  
  
int main() {  
    Derivata d_obj;  
    Base* b_ptr = &d_obj;  
  
    // Punto L
```

```
Derivata* d_ptr = b_ptr;  
  
d_ptr->special();  
}
```

Cosa succede al Punto L, dove si tenta un **downcasting implicito**?

- A. L'assegnazione è valida e il codice funziona. B.
- B. Errore di compilazione. Il downcasting (da Base a Derivata) non è consentito implicitamente e richiede un cast esplicito (e insicuro).
- C. L'assegnazione è valida solo se **Base** ha un metodo **virtual**.
- D. Il codice funziona, ma se **b_ptr** non puntasse effettivamente a **Derivata** causerebbe un errore di runtime.

Risposta Corretta: B

L'**Upcasting** (Da Derivata a Base) è sempre sicuro e implicito. Il **Downcasting** (Da Base a Derivata) è **intrinsecamente non sicuro** (il puntatore Base potrebbe non puntare a una Derivata completa) e quindi richiede un cast esplicito (come **static_cast**, **dynamic_cast**, o C-style cast).

Domanda 17

Quale tipo di ereditarietà (pubblica, protetta o privata) è **necessario** affinché un Upcasting (assegnare un puntatore/riferimento di **Derivata** a un puntatore/riferimento di **Base**) sia consentito?

- A. Ereditarietà **protected** o **public**.
- B. Solo ereditarietà **protected**.
- C. Ereditarietà **private** o **public**.
- D. Solo ereditarietà **public**.

Risposta Corretta: D

L'**Upcasting** (considerare un oggetto derivato come base) è la base del polimorfismo ed è consentito **solo** se l'ereditarietà è **public**. L'ereditarietà **protected** o **private** interrompe la relazione "è-un" per il codice esterno e non consente l'Upcasting implicito.